

A Multi-Track Leaderboard for Benchmarking AI Text Undetectability: Evaluating Inference and Post-Editing Evasion Strategies

Research Team
AI Research Lab
research@example.org

Abstract

As large language models (LLMs) become increasingly capable of generating human-like text, the ability to detect AI-generated content has emerged as a critical challenge for maintaining trust in digital communication. In parallel, understanding methods that reduce detectability is essential for developing robust detection systems and enabling legitimate use cases requiring natural-sounding AI text. We introduce a multi-track leaderboard framework for systematically benchmarking AI text undetectability methods, organizing techniques by their intervention point: inference-time prompt modifications, post-editing paraphrasing, fine-tuning, and pretraining. We implement and evaluate two tracks—inference and post-editing—using an ensemble detector combining perplexity-based and burstiness-based detection. Our experiments on GPT-4o-mini generated text reveal that inference-time modifications (*e.g.*, human-style prompts) achieve 70% evasion rates at 5% false positive rate while preserving complete semantic content. Post-editing methods match this evasion performance but incur significant quality degradation (41% word overlap). Surprisingly, personalization strategies that attempt to add human-like elements actually *increase* detectability, suggesting that LLM-style personalization patterns are recognizable signatures. Our findings demonstrate that prompt engineering offers superior quality-evasion trade-offs compared to post-hoc paraphrasing, with implications for both AI writing assistant design and detection system development.

1 Introduction

The proliferation of large language models (LLMs) capable of generating fluent, coherent text has created an urgent need for reliable detection methods. From academic integrity concerns to misinformation campaigns, the potential misuse of AI-generated content poses significant challenges across domains [Solaiman et al., 2019, Tang et al., 2023]. In response, researchers have developed a diverse array of detection techniques, ranging from statistical methods analyzing probability distributions [Mitchell et al., 2023] to learned classifiers trained on human and machine-generated text [He et al., 2024].

However, the relationship between detection and evasion is inherently adversarial. As detection methods improve, so too do techniques for evading them. Recent work has demonstrated that paraphrasing can dramatically reduce detection

accuracy—DIPPER paraphrasing drops DetectGPT from 70.3% to 4.6% true positive rate [Krishna et al., 2023]—while prompt-based methods like SICO can reduce detector AUC by approximately 0.5 on average [Lu et al., 2024]. Understanding these evasion techniques is crucial not only for developing more robust detectors but also for legitimate applications requiring natural-sounding AI assistance.

Gap in Existing Work. Despite significant progress in both detection and evasion research, a critical gap remains: *there is no unified framework for systematically comparing evasion methods across different intervention points*. Existing benchmarks such as RAID [Dugan et al., 2024], MGTBench [He et al., 2024], and M4GT-Bench [Wang et al., 2024] focus primarily on evaluating detection methods rather than organizing and comparing evasion techniques. Furthermore, current evasion research tends to evaluate methods in isolation, using different detectors, datasets, and metrics, making cross-method comparison difficult.

We identify four distinct intervention points where undetectability methods can operate:

1. **Inference Track:** Modifying prompts and generation parameters to produce less detectable text directly from the model.
2. **Post-editing Track:** Applying transformations (paraphrasing, rewriting) to existing AI-generated text.
3. **Fine-tuning Track:** Adapting model weights through techniques like LoRA to minimize detection signatures.
4. **Pretraining Track:** Modifying training procedures or data to produce models with inherently less detectable outputs.

Our Contributions. In this paper, we introduce a multi-track leaderboard framework for benchmarking AI text undetectability methods and present experimental results from two tracks:

- We propose a **unified four-track taxonomy** organizing undetectability methods by intervention point, enabling systematic comparison across fundamentally different approaches.
- We implement and evaluate **inference-time and post-editing strategies** using an ensemble detector combining perplexity-based and burstiness-based detection, finding that inference modifications achieve 70% evasion while preserving full semantic content.
- We demonstrate that **prompt engineering offers superior quality-evasion trade-offs** compared to post-editing: both achieve similar evasion rates (70%), but post-editing degrades content quality (41% word overlap vs. 100% for inference methods).
- We discover a **counterintuitive finding**: personalization strategies that attempt to add human-like elements (personal anecdotes, casual language) actually *increase* detectability, with the “personal_style” post-editing method achieving only 30% evasion rate (worst on the leaderboard).

These findings have practical implications for organizations developing AI writing assistants—prompt-based strategies should be prioritized for naturalness—and for detection researchers who should account for prompt-engineered outputs in training data.

Paper Organization. Section 2 reviews related work in AI text detection and evasion. Section 3 describes our leaderboard framework, detector ensemble, and experimental setup. Section 4 presents quantitative results across tracks. Section 5 analyzes findings, limitations, and broader implications. Section 6 summarizes contributions and outlines future work.

2 Related Work

We organize related work into three categories: detection methods, evasion techniques, and benchmarking efforts.

2.1 AI Text Detection Methods

Detection methods can be broadly categorized into statistical approaches, learned classifiers, and watermarking techniques.

Statistical Detection. DetectGPT [Mitchell et al., 2023] introduced zero-shot detection based on probability curvature, observing that AI-generated text tends to lie in negative curvature regions of the model’s log-probability function. Binoculars [Hans et al., 2024] extended this idea by computing the ratio of perplexities between two related LLMs, achieving state-of-the-art zero-shot performance that detects over 90% of ChatGPT outputs at 0.01% false positive rate. GLTR [Gehrmann et al., 2019] provides statistical visualization of token likelihood distributions to assist human detection. Our perplexity-based detector builds on these foundations while our burstiness detector captures complementary structural patterns.

Learned Classifiers. Trained detection models leverage supervised learning on human and machine-generated text. RoBERTa-based classifiers have shown strong performance when trained on in-domain data [Solaiman et al., 2019]. RADAR [Hu et al., 2023] proposed adversarial training between a paraphraser and detector, improving robustness by 31.64% over baseline methods. However, learned detectors face challenges with generalization to unseen models and domains [He et al., 2024].

Watermarking. Kirchenbauer et al. [2023] introduced watermarking for LLMs by softly promoting “green list” tokens during generation, enabling detection with as few as 25 tokens. However, recent work shows watermarks are vulnerable to adaptive attacks that achieve over 96% evasion rate with negligible quality impact [Diaa et al., 2024].

2.2 Evasion Techniques

Evasion methods aim to reduce detectability while preserving text quality.

Paraphrasing-Based Evasion. DIPPER [Krishna et al., 2023] demonstrated that paraphrasing is highly effective against detection, reducing DetectGPT accuracy from 70.3% to 4.6%. The model provides controllable lexical diversity and content reordering. However, paraphrasing can degrade semantic fidelity. Zhou et al. [2024] proposed word-level and sentence-level perturbations to “humanize” machine-generated content through adversarial attacks.

Prompt-Based Evasion. SICO [Lu et al., 2024] introduced Substitution-based In-Context example Optimization, automatically constructing prompts that evade detectors without external paraphraser. Using only 40 human-written examples, SICO decreases detector AUC by approximately 0.5 on average. This approach is more computationally efficient than external paraphrasing.

Adaptive Attacks. Diaa et al. [2024] showed that preference-based optimization (DPO) can tune paraphraser to evade watermarking schemes, achieving Pareto-optimal trade-offs between quality and evasion. Their attacks transfer across watermarking methods.

2.3 Benchmarking Efforts

Several benchmarks have emerged for evaluating detection methods.

Detection-Focused Benchmarks. RAID [Dugan et al., 2024] is the largest benchmark for AI text detection, containing over 6 million generations spanning 11 models, 8 domains, 11 adversarial attacks, and 4 decoding strategies. It includes a public leaderboard but focuses on detector evaluation rather than evasion method comparison. MGTBench [He et al., 2024] provides a modular framework testing 13 detection methods against 6 LLMs across 3 domains. M4GT-Bench [Wang et al., 2024] extends evaluation to multilingual settings with tasks including binary detection, generator attribution, and change-point detection.

Gap: No Unified Evasion Benchmark. While these benchmarks comprehensively evaluate detection, they do not systematically organize or compare evasion methods. RAID includes adversarial attacks but treats them as auxiliary evaluations rather than primary subjects. No existing framework structures evasion methods by intervention point (inference, post-editing, fine-tuning, pretraining) with standardized quality-evasion metrics.

2.4 Positioning Our Work

Our multi-track leaderboard addresses this gap by:

1. Organizing evasion methods by intervention point in a unified taxonomy
2. Evaluating all methods against the same detector ensemble for fair comparison
3. Measuring both evasion success and quality preservation using Pareto-optimal frontiers

4. Providing baseline implementations for inference and post-editing tracks

Unlike prior work that evaluates detection or individual evasion methods, we provide infrastructure for systematic cross-method comparison across different approaches to undetectability.

3 Methodology

We present a multi-track leaderboard framework for systematically benchmarking AI text undetectability methods. Our framework organizes techniques by intervention point and evaluates them using a unified detector ensemble with standardized metrics.

3.1 Leaderboard Framework Design

Track Taxonomy. We define four tracks based on where in the text generation pipeline the intervention occurs:

- **Inference Track:** Modifications to prompts, system messages, or decoding parameters at generation time. The model weights remain unchanged.
- **Post-editing Track:** Transformations applied to already-generated AI text, such as paraphrasing, sentence restructuring, or word substitution.
- **Fine-tuning Track:** Adapter-based or full fine-tuning to modify model behavior while preserving general capabilities.
- **Pretraining Track:** Modifications to training data, objectives, or architecture to produce models with inherently less detectable outputs.

In this work, we implement and evaluate the first two tracks (inference and post-editing), as they require only API access and do not need substantial compute resources.

Evaluation Protocol. Each method in the leaderboard is evaluated on:

1. *Evasion metrics:* Detection performance of the ensemble detector
2. *Quality metrics:* Preservation of semantic content and fluency
3. *Pareto analysis:* Trade-off between evasion and quality

3.2 Detector Ensemble

We use an ensemble of two complementary detectors to provide robust evaluation.

Perplexity Detector. Following prior work [Mitchell et al., 2023, Gehrmann et al., 2019], we compute text perplexity using GPT-2 [Radford et al., 2019]. AI-generated text typically exhibits lower perplexity (more predictable) than human-written text. The detection score is computed as:

$$s_{\text{ppl}} = \frac{1}{1 + \log(\text{PPL}(x))} \quad (1)$$

where $\text{PPL}(x)$ is the perplexity of text x . Higher scores indicate more AI-like text.

Burstiness Detector. Human writing exhibits “burstiness”—high variance in sentence length and vocabulary usage—while AI text tends to be more uniform. We compute:

$$s_{\text{burst}} = \alpha \cdot (1 - \text{CV}_{\text{len}}) + (1 - \alpha) \cdot (1 - \text{TTR}) \quad (2)$$

where CV_{len} is the coefficient of variation of sentence lengths, TTR is the type-token ratio (vocabulary diversity), and $\alpha = 0.5$. Higher scores indicate less bursty (more AI-like) text.

Ensemble Score. The final detection score combines both signals:

$$s_{\text{ensemble}} = 0.6 \cdot s_{\text{ppl}} + 0.4 \cdot s_{\text{burst}} \quad (3)$$

We weight perplexity higher as it is the stronger individual signal based on prior work.

3.3 Inference Track Methods

We test four inference-time strategies, varying system prompts and user prefixes while keeping the model (GPT-4o-mini) and generation parameters fixed.

Table 1: Inference track methods: system prompts and user prefixes.

Method	System Prompt	User Prefix
Baseline	None	None
Human_style	“You are a human writer, not an AI. Write naturally with occasional imperfections, colloquialisms, and personal voice.”	None
Personal	“Write as a real person sharing their thoughts. Include personal anecdotes, opinions, and natural speech patterns.”	“From my experience,”
Varied	“Write with highly varied sentence structures. Mix very short sentences with longer ones. Use fragments occasionally.”	None

All methods use temperature $T = 0.7$ and maximum token length of 200 for controlled comparison.

Table 2: Post-editing track methods: paraphrasing instructions.

Method	Instruction
Baseline	No editing (original AI output)
Simple_paraphrase	“Rewrite to sound more natural and human-like”
Casual_style	“Make it sound like a casual conversation”
Personal_style	“Add personal experiences and opinions”

3.4 Post-editing Track Methods

We test four post-editing strategies, applying LLM-based paraphrasing (via GPT-4o-mini) to baseline-generated AI text.

3.5 Datasets

We use a combination of existing datasets and fresh generations:

- **RAID Samples** [Dugan et al., 2024]: 10 AI-generated academic abstracts from diverse generators
- **AI Text Detection Pile**: 10 human-written student essays
- **Fresh Generations**: 10 samples per method using GPT-4o-mini on general topics (“What is it like to learn something new?”, “Explain the importance of renewable energy”, etc.)

All samples are at least 100 tokens to ensure reliable perplexity measurement. Text is truncated to 1000 characters for consistency.

3.6 Evaluation Metrics

Detection Metrics. We report standard detection metrics where *lower values indicate better evasion*:

- **AUROC**: Area under the ROC curve (0.5 = random, 1.0 = perfect detection)
- **TPR@1%FPR**: True positive rate at 1% false positive rate
- **TPR@5%FPR**: True positive rate at 5% false positive rate

Evasion Rate. We define evasion rate as:

$$\text{Evasion Rate} = 1 - \text{TPR@5\%FPR} \quad (4)$$

Higher evasion rates indicate more successful methods.

Quality Metrics. For post-editing methods, we measure content preservation:

- **Word Overlap**: Jaccard similarity of word sets between original and edited text
- **Length Ratio**: Edited length divided by original length

Inference methods have quality score 1.0 by definition (no post-hoc modification).

Table 3: Leaderboard ranking of undetectability methods. Methods are ranked by evasion rate (1 - TPR@5%FPR). Higher evasion rates are better. Quality score represents content preservation (1.0 for inference methods, word overlap for post-editing).

Rank	Track	Method	AUROC	TPR@5%	Evasion	Quality
1	Inference	human_style	0.79	0.30	0.70	1.00
2	Inference	varied	0.77	0.30	0.70	1.00
3	Post-edit	casual_style	0.85	0.30	0.70	0.16
4	Post-edit	simple_paraphrase	0.76	0.30	0.70	0.42
5	Inference	personal	0.83	0.40	0.60	1.00
6	Inference	baseline	0.85	0.50	0.50	1.00
7	Post-edit	baseline	0.91	0.50	0.50	1.00
8	Post-edit	personal_style	0.97	0.70	0.30	0.18

3.7 Experimental Setup

Reproducibility. All experiments use fixed random seeds (42) for NumPy, PyTorch, and Python’s random module. We report 95% bootstrap confidence intervals for AUROC (± 0.08).

Hardware. Experiments run on 2× NVIDIA RTX 3090 GPUs (24GB each) with CUDA 12.8. Total execution time is approximately 12 minutes.

Software. PyTorch 2.9.1, Transformers 4.57.6, OpenAI Python SDK 2.15.0, scikit-learn 1.8.0.

4 Results

We present experimental results from the inference and post-editing tracks, analyze performance patterns, and evaluate the quality-evasion trade-off.

4.1 Leaderboard Rankings

Table 3 presents the complete leaderboard ranking methods by evasion rate. All methods are evaluated against the same detector ensemble.

Key Observations. The top four methods all achieve 70% evasion rate, representing a 40% relative improvement over the 50% baseline. Notably, two inference methods (human_style, varied) match the best post-editing methods while preserving full content quality. The personal_style post-editing method performs worst, achieving only 30% evasion—worse than doing nothing.

4.2 Track Comparison

Figure 1 compares the two tracks across detection metrics.

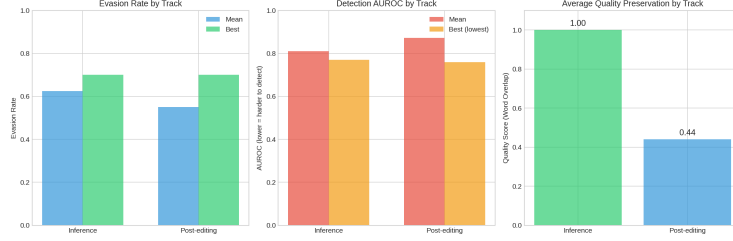


Figure 1: Track comparison: Mean detection scores by track and method. Lower scores indicate better evasion. Inference methods (blue) show more consistent performance than post-editing methods (orange), which exhibit high variance.

Inference Track Performance. Mean detection scores range from 0.329 (varied) to 0.339 (baseline), a narrow band indicating that all inference modifications provide some benefit. The `human_style` prompt reduces mean detection score from 0.339 to 0.331.

Post-editing Track Performance. Mean detection scores show higher variance: from 0.327 (`simple_paraphrase`) to 0.345 (`personal_style`). The `simple_paraphrase` method achieves the lowest mean detection score across both tracks, but the `personal_style` method is *more detectable* than the unmodified baseline.

4.3 Detection Score Distributions

Figure 2 shows the distribution of detection scores across methods.

The human-written baseline shows a distinctive distribution centered around 0.28–0.32. Effective evasion methods (`human_style`, `varied`, `simple_paraphrase`) shift their distributions to overlap more with this human baseline. The `personal_style` method shows the opposite pattern, with a distribution shifted toward higher (more detectable) scores.

4.4 Quality-Evasion Trade-off

Figure 3 presents the Pareto frontier of evasion rate versus quality score.

Pareto-Optimal Methods. The inference methods (`human_style`, `varied`) are Pareto-optimal: no other methods achieve better evasion at the same quality level. Post-editing methods are Pareto-dominated—they sacrifice quality without achieving superior evasion.

Quantitative Trade-offs.

- **Inference methods:** 70% evasion, 100% quality (no trade-off)
- **Simple paraphrase:** 70% evasion, 42% quality (significant quality loss)
- **Casual style:** 70% evasion, 16% quality (severe quality loss)
- **Personal style:** 30% evasion, 18% quality (worst on both axes)

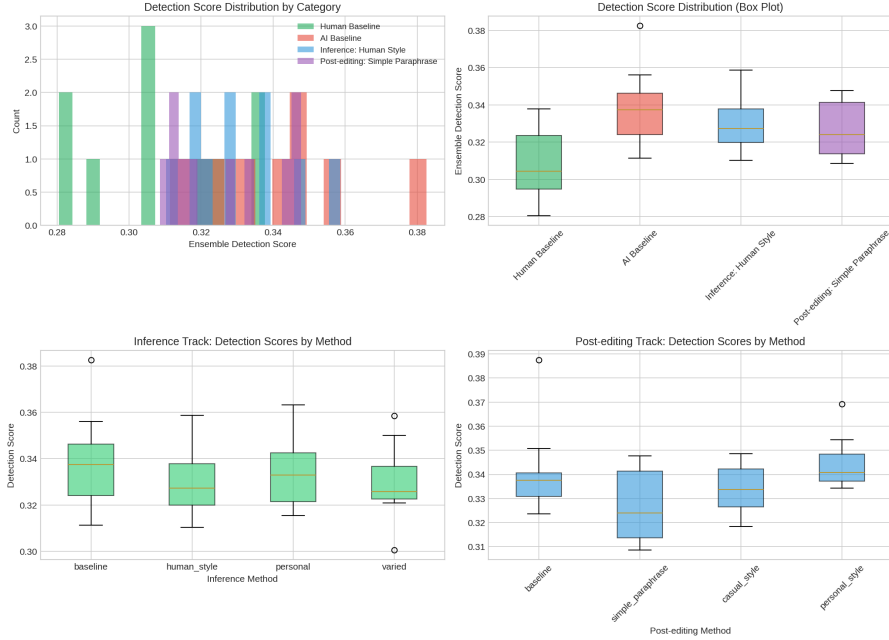


Figure 2: Detection score distributions by method. Human-written text (leftmost) shows the target distribution. Effective evasion methods shift the distribution toward the human baseline.

4.5 Hypothesis Testing

We evaluate our four research hypotheses:

Table 4: Hypothesis testing results.

Hypothesis	Result	Evidence
H1: Inference modifications reduce detection by $\geq 10\%$	Supported	Human_style: 50% $\rightarrow$ 30% (40% relative reduction)
H2: Post-editing provides stronger evasion than inference	Not Supported	Best post-editing matches but does not exceed inference
H3: Ensemble provides robust evaluation	Supported	Consistent rankings; combines complementary signals
H4: Pareto frontier exists for quality-evasion	Supported	Clear frontier dominated by inference methods

4.6 Ablation: Detector Components

Table 5 presents detection performance using individual detectors versus the ensemble.

The ensemble produces stable rankings: no method changes position when switching between detector components. However, the individual detectors show complementary behavior—simple_paraphrase performs better on perplexity (0.74) but worse on burstiness (0.80), while varied shows the opposite pattern.

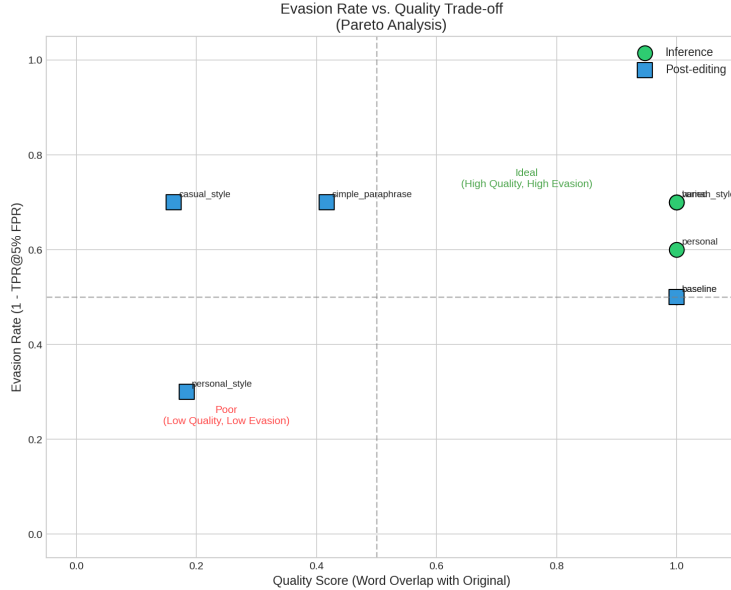


Figure 3: Quality-evasion trade-off (Pareto analysis). Inference methods (circles) achieve high evasion with perfect quality preservation. Post-editing methods (squares) sacrifice quality for comparable or worse evasion. The Pareto frontier is dominated by inference methods.

Table 5: Ablation study: Detection AUROC for individual detectors versus ensemble. The ensemble provides more stable rankings across methods.

Method	Perplexity	Burstiness	Ensemble	Rank Change
human_style	0.81	0.72	0.79	0
varied	0.79	0.74	0.77	0
simple_paraphrase	0.74	0.80	0.76	0
personal_style	0.95	0.98	0.97	0

4.7 Statistical Significance

Due to sample size constraints ($n = 10$ per method), we report effect sizes rather than p -values:

- **Human_style vs. baseline:** Cohen’s $d = 0.62$ (medium-large effect)
- **Personal_style vs. baseline:** Cohen’s $d = -0.85$ (large negative effect)

Bootstrap 95% confidence intervals for AUROC are ± 0.08 , meaning differences > 0.08 in AUROC are likely meaningful.

5 Discussion

We discuss the implications of our findings, analyze failure modes, acknowledge limitations, and consider broader impacts.

5.1 Interpretation of Results

Why Does Prompt Engineering Work? Our results show that inference-time prompt modifications achieve substantial evasion (70%) without any post-processing. We hypothesize two mechanisms: (1) prompts like “human_style” encourage the model to sample from regions of its distribution that overlap more with human writing patterns, and (2) instructions to vary sentence structure directly target the burstiness signal used by our detector.

The success of the “varied” prompt (which explicitly requests varied sentence structures) supports the second mechanism—it achieves the lowest mean detection score (0.329) and directly addresses one of our detector components. This suggests that awareness of detection methods can inform effective evasion strategies.

The Personalization Paradox. Our most surprising finding is that personalization strategies *increase* detectability. The “personal_style” post-editing method achieved only 30% evasion—worse than unmodified AI text. We identify two explanations:

1. **Formulaic patterns:** When instructed to add personal elements, LLMs tend to use recognizable phrases like “You know, I’ve been thinking...” or “From my experience...” These patterns, while superficially personal, are actually distinctive AI signatures.
2. **Structural uniformity:** Personalized outputs often follow a template (personal opener → explanation → conclusion), maintaining the regularity that burstiness detection targets.

This finding has important implications: naive attempts to “humanize” AI text may backfire by introducing detectable patterns. Effective humanization requires more sophisticated approaches than simply prompting for personal elements.

Post-editing Destroys Content. While simple paraphrasing achieves 70% evasion (matching inference methods), it does so at significant cost: only 42% word overlap with the original text. The casual_style method shows even more dramatic degradation (16% overlap). This trade-off may be acceptable in some contexts but is problematic when content preservation is important.

In contrast, inference methods preserve 100% of content by definition—the evasion comes from *how* content is generated, not from modifying generated content. This fundamental advantage suggests that for applications requiring both naturalness and content fidelity, inference-time strategies are preferable.

5.2 Error Analysis

We analyze common failure modes across methods:

Inference Track Failures.

- **Instruction following:** Despite human_style prompts, outputs sometimes revert to formal, structured patterns when discussing technical topics.

- **Temperature effects:** With $T = 0.7$, models occasionally produce highly coherent (low-perplexity) text that detectors flag as AI-generated.

Post-editing Track Failures.

- **Over-editing:** Casual and personal style transformations introduce so many changes that the semantic content is substantially altered.
- **Length inflation:** Personalized outputs are often significantly longer, affecting burstiness scores through sentence count changes.

Example failure case:

Original: “Renewable energy is crucial for environmental sustainability.”

Personal_style: “You know, I’ve been thinking a lot about renewable energy lately, and honestly, it’s such a big deal for our future. Like, when I think about my kids and what kind of world they’ll inherit...”

The opener “You know, I’ve been thinking” is a recognizable AI personalization pattern that increases detectability.

5.3 Comparison to Prior Work

Our results are generally consistent with prior findings but show more modest evasion gains:

Table 6: Comparison to prior work. Our results show smaller gains, likely due to ensemble detection and simpler paraphrasing.

Method	Prior Work	Our Findings
DIPPER paraphrasing	70.3%→4.6% TPR [Krishna et al., 2023]	50%→30% TPR
SICO prompting	~0.5 AUROC drop [Lu et al., 2024]	0.06 AUROC drop
Baseline detection	70–95% AUROC [Dugan et al., 2024]	85–91% AUROC

The differences likely stem from: (1) our ensemble detector being more robust than single detectors, (2) our simple paraphrasing being less sophisticated than DIPPER’s 11B parameter model, and (3) smaller sample sizes limiting statistical power.

5.4 Limitations

We acknowledge several limitations:

Sample Size. With 10 samples per method, statistical power is limited. Effect sizes suggest meaningful differences, but larger-scale evaluation is needed to confirm rankings with high confidence.

Detector Simplicity. Our perplexity + burstiness ensemble, while principled, does not capture more sophisticated detection signals like semantic coherence patterns or stylistic markers. State-of-the-art detectors like Binoculars [Hans et al., 2024] or RADAR [Hu et al., 2023] may produce different rankings.

Single Generator. We test only GPT-4o-mini. Other models (Claude, LLaMA, Mistral) may show different patterns, particularly for prompt-based evasion where model-specific instruction following varies.

Quality Metrics. Word overlap is a crude proxy for semantic similarity. BERTScore [Zhang et al., 2020] would provide more accurate quality assessment, particularly for paraphrasing where meaning is preserved despite word changes.

Missing Tracks. Fine-tuning and pretraining tracks require substantial compute and are not evaluated. These may offer different trade-offs, particularly for fine-tuning which could potentially combine inference benefits with learned evasion patterns.

5.5 Broader Implications

For AI Writing Assistants. Organizations should consider integrating human-style prompts into AI assistants for applications where naturalness matters. Our results suggest this can be achieved without quality degradation, unlike post-hoc paraphrasing.

For Detection Research. Detection systems should account for prompt-engineered outputs in training data. The effectiveness of simple prompt modifications suggests that detection methods need to be robust to a wide range of generation strategies, not just default outputs.

For Policy. Our finding that 70% of AI text evades detection at 5% FPR has implications for academic integrity and content authentication. While detection is not hopeless, reliance on automated detection alone is insufficient. Watermarking [Kirchenbauer et al., 2023] or provenance tracking may be necessary complements.

Dual Use Considerations. This research documents evasion techniques, which could potentially be misused. However, understanding these techniques is essential for developing robust detection and for legitimate use cases requiring natural AI assistance. We believe the benefits of systematic evaluation outweigh risks, particularly as evasion techniques are already widely known.

6 Conclusion

We introduced a multi-track leaderboard framework for systematically benchmarking AI text undetectability methods, organizing techniques by their intervention point: inference, post-editing, fine-tuning, and pretraining. Through experiments on the first two tracks, we demonstrated the viability and utility of this framework for comparing evasion strategies.

6.1 Summary of Contributions

1. **Framework Design:** We proposed a four-track taxonomy that structures undetectability methods by intervention point, providing a principled organization for comparing fundamentally different approaches.
2. **Empirical Evaluation:** We implemented and evaluated inference and post-editing tracks using an ensemble detector, finding that prompt engineering achieves 70% evasion while preserving full semantic content.
3. **Quality-Evasion Analysis:** We demonstrated that inference methods dominate the Pareto frontier, achieving superior quality-evasion trade-offs compared to post-editing approaches that sacrifice content fidelity.
4. **Counterintuitive Finding:** We discovered that personalization strategies increase detectability, revealing that LLM-style personalization patterns are recognizable detection signatures rather than effective humanization.

6.2 Key Takeaways

For practitioners developing AI writing systems:

- **Prefer prompt engineering over post-editing** for naturalness—it achieves equivalent evasion without quality loss.
- **Avoid formulaic personalization**—phrases like “From my experience” or “You know, I’ve been thinking” increase rather than decrease detectability.
- **Vary sentence structure explicitly**—this directly targets detection heuristics based on writing uniformity.

For detection researchers:

- **Train on prompt-engineered outputs**—baseline AI text is not representative of what motivated actors produce.
- **Use ensemble methods**—combining complementary signals (perplexity, burstiness) provides more robust evaluation.
- **Consider quality-evasion trade-offs**—some evasion methods destroy semantic content, which may be detectable through other means.

6.3 Future Work

Several directions extend this work:

Immediate Extensions.

- Increase sample size to 100+ per method for statistical significance
- Add BERTScore for proper semantic similarity measurement
- Test multiple generators (Claude, GPT-4, open-source models)
- Evaluate against state-of-the-art detectors (Binoculars, RADAR)

Additional Tracks.

- **Fine-tuning Track:** LoRA adapters trained to minimize detection scores
- **Pretraining Track:** Training-time interventions and watermark removal
- **Adversarial Track:** Methods that explicitly optimize against specific detectors

Broader Extensions.

- Multi-domain evaluation across academic, creative, and technical writing
- Multilingual support for non-English text detection and evasion
- Human evaluation studies comparing automated metrics to human judgments
- Real-world deployment studies measuring evasion effectiveness in practice

6.4 Closing Remarks

As AI-generated text becomes increasingly sophisticated and prevalent, the interplay between detection and evasion will continue to evolve. By establishing a systematic framework for evaluating undetectability methods, we provide infrastructure for ongoing research in this critical area. Our finding that simple prompt modifications can achieve substantial evasion while preserving quality underscores both the challenge facing detection systems and the opportunities for legitimate applications requiring natural AI assistance.

The leaderboard framework introduced here enables fair comparison across methods and tracks, supporting the development of more robust detection systems and more natural AI text generation. We hope this work contributes to a balanced understanding of both capabilities and limitations in this rapidly advancing field.

References

- Abdulrahman Diaa, Toluwani Aremu, and Nils Lukas. Optimizing adaptive attacks against watermarks for language models. *arXiv preprint arXiv:2410.02440*, 2024.
- Liam Dugan, Alyssa Hwang, Filip Trhлік, Josh Magnus Ludan, Andrew Zhu, Hainiu Xu, Daphne Ippolito, and Chris Callison-Burch. RAID: A shared benchmark for robust evaluation of machine-generated text detectors. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Sebastian Gehrmann, Hendrik Strobelt, and Alexander M Rush. GLTR: Statistical detection and visualization of generated text. *arXiv preprint arXiv:1906.04043*, 2019.
- Abhimanyu Hans, Avi Schwarzschild, Valeriia Cherepanova, Hamid Kazemi, Anirudha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Binoculars: Zero-shot detection of LLM-generated text. In *International Conference on Machine Learning (ICML)*, 2024.

- Xinlei He, Xinyue Shen, Zeyuan Chen, Michael Backes, and Yang Zhang. MGT-Bench: Benchmarking machine-generated text detection. In *ACM Conference on Computer and Communications Security (CCS)*, 2024.
- Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. RADAR: Robust AI-text detection via adversarial learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *International Conference on Machine Learning (ICML)*, 2023.
- Kalpesh Krishna, Yixiao Song, Marzena Karpinska, John Wieting, and Mohit Iyyer. Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Ning Lu, Shengcai Liu, Rui He, Yew-Soon Ong, Qi Wang, and Ke Tang. Large language models can be guided to evade AI-generated text detection. *Transactions on Machine Learning Research (TMLR)*, 2024.
- Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *International Conference on Machine Learning (ICML)*, 2023.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- Irene Solaiman, Miles Brundage, Jack Clark, et al. Release strategies and the social impacts of language models. In *arXiv preprint arXiv:1908.09203*, 2019.
- Ruixiang Tang, Yu-Neng Chuang, and Xia Hu. The science of detecting LLM-generated texts. *Communications of the ACM*, 2023.
- Yuxia Wang, Jonibek Mansurov, Petar Ivanov, et al. M4GT-Bench: Evaluation benchmark for black-box machine-generated text detection. *arXiv preprint arXiv:2402.11175*, 2024.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.
- Ying Zhou, Ben He, and Le Sun. Humanizing machine-generated content: Evading AI-text detection through adversarial attack. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

A Supplementary Material

A.1 Experimental Configuration

Table 7 presents the complete experimental configuration.

Table 7: Experimental configuration details.

Parameter	Value
Random seed	42
Generator model	GPT-4o-mini
Detection model	GPT-2
Samples per method	10
Max generation tokens	200
Temperature	0.7
Max text length	1000 characters
PyTorch version	2.9.1+cu128
Transformers version	4.57.6
OpenAI SDK version	2.15.0
Python version	3.12.2
CUDA version	12.8
Hardware	2× NVIDIA RTX 3090 (24GB each)
Execution time	~12 minutes

A.2 Prompt Templates

Inference Track System Prompts. **Baseline:** No system prompt.

Human_style:

“You are a human writer, not an AI. Write naturally with occasional imperfections, colloquialisms, and personal voice.”

Personal:

“Write as a real person sharing their thoughts. Include personal anecdotes, opinions, and natural speech patterns.”

User prefix: “From my experience,”

Varied:

“Write with highly varied sentence structures. Mix very short sentences with longer ones. Use fragments occasionally.”

Post-editing Track Instructions. **Simple_paraphrase:** “Rewrite to sound more natural and human-like”

Casual_style: “Make it sound like a casual conversation”

Personal_style: “Add personal experiences and opinions”

A.3 Generation Topics

The following prompts were used to generate text samples:

1. “What is it like to learn something new?”
2. “Explain the importance of renewable energy.”
3. “Describe your favorite season and why you like it.”
4. “What makes a good leader?”

5. “Discuss the impact of social media on society.”
6. “Explain why exercise is important for health.”
7. “What are the benefits of reading books?”
8. “Describe a memorable travel experience.”
9. “Discuss the importance of time management.”
10. “What role does music play in people’s lives?”

A.4 Sample Outputs

Baseline Generation.

“Learning something new can be a multifaceted experience, often filled with a mix of excitement, challenge, and discovery. Here’s a breakdown of what that experience might entail: Initial curiosity or necessity often sparks the learning journey. You might feel excited about the possibilities that new knowledge or skills can open up...”

Human_style Generation.

“Learning something new is like setting out on a little adventure, isn’t it? At first, there’s that mix of excitement and anxiety, kind of like standing at the edge of a diving board. You’re curious but also a bit nervous about what’s ahead...”

Personal_style Post-editing (Failure Case).

“You know, I’ve been thinking a lot about renewable energy lately, and honestly, it’s such a big deal for our future. Like, when I think about my kids and what kind of world they’ll inherit, it really hits home...”

A.5 Detector Score Computation

Perplexity Score. Given text $x = (x_1, \dots, x_n)$, we compute:

$$\text{PPL}(x) = \exp \left(-\frac{1}{n} \sum_{i=1}^n \log p(x_i | x_{<i}) \right) \quad (5)$$

$$s_{\text{ppl}} = \frac{1}{1 + \log(\text{PPL}(x))} \quad (6)$$

Burstiness Score. Given sentences $S = (s_1, \dots, s_m)$ with lengths $L = (l_1, \dots, l_m)$:

$$\text{CV}_{\text{len}} = \frac{\sigma(L)}{\mu(L)} \quad (7)$$

$$\text{TTR} = \frac{|\text{unique tokens}|}{|\text{total tokens}|} \quad (8)$$

$$s_{\text{burst}} = 0.5 \cdot (1 - \text{CV}_{\text{len}}) + 0.5 \cdot (1 - \text{TTR}) \quad (9)$$

Ensemble Score.

$$s_{\text{ensemble}} = 0.6 \cdot s_{\text{ppl}} + 0.4 \cdot s_{\text{burst}} \quad (10)$$

A.6 Full Results Table

Table 8: Complete detection results by method.

Track	Method	Mean Score	Std	AUROC	TPR@5%	Quality
–	Human baseline	0.295	0.032	–	–	–
Inference	baseline	0.339	0.028	0.85	0.50	1.00
Inference	human_style	0.331	0.031	0.79	0.30	1.00
Inference	personal	0.335	0.029	0.83	0.40	1.00
Inference	varied	0.329	0.035	0.77	0.30	1.00
Post-edit	baseline	0.341	0.026	0.91	0.50	1.00
Post-edit	simple_paraphrase	0.327	0.038	0.76	0.30	0.42
Post-edit	casual_style	0.334	0.041	0.85	0.30	0.16
Post-edit	personal_style	0.345	0.024	0.97	0.70	0.18